TD/TP2 : Diviser pour régner, récursivité et complexité

CPES « Sciences des données, arts et cultures »

Antoine Gauquier
antoine.gauquier@ens.fr

Pierre Aboulker
pierreaboulker@gmail.com

06 octobre 2025

Dans ce TD/TP, nous nous intéressons au paradigme « diviser pour régner », au calcul de la complexité temporelle d'algorithmes récursifs et à l'implémentation de tels algorithmes en Python.

Exercice 1 : Notations asymptotiques de fonctions récursives

Dans cet exercice, il vous est demandé de noter la notation asymptotique $\Theta(\cdot)$ associée à chacune des expressions de récurrence présentées ci-dessous. Pour chaque cas, la méthode à employer est indiquée. Trois possibilités :

- Substitution. En partant d'une hypothèse sur la notation, démontrez qu'elle est vraie.
- Arbre de récurrence. Ecrire l'arbre de récurrence à partir de l'expression $T(n)$ fournie, et conclure sur la complexité, en calculant la somme des complexités de chaque niveau de l'arbre.
- Théorème maître. Donner les paramètres du théorème maître, puis, à partir de ceux-ci, justifier la complexité.

Question 1 $T(n) = T\left(\frac{n}{2}\right) + 1$. Substitution.

Question 2 $T(n) = 4T\left(\frac{n}{2}\right) + n$. Arbre de récurrence.

Question 3 $T(n) = 2T\left(\frac{n}{2}\right) + n^3$. Théorème maître.

Question 4 $T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n}$. Théorème maître.

Question 5 $T(n) = 4T\left(\frac{n}{2}\right) + n\log n$. Théorème maître.

Question 6 $T(n) = 4T\left(\frac{3n}{4}\right) + 5$. Théorème maître.

Exercice 2 : Recherche dichotomique

Question 1 Donner la relation de récurrence du nombre d'opérations élémentaires dans la recherche dichotomique $T(n)$. Justifier la relation obtenue.

Question 2 A partir de la relation obtenue, construire un arbre de récurrence des appels successifs à T. Déterminer la complexité asymptotique en $\Theta(\cdot)$, en analysant le nombre d'opérations décrites par l'arbre.

Question 3 Vérifier la complexité asymptotique obtenue en utilisant le théorème maître.

Exercice 3 : Tri fusion en Python

Dans cette partie « TP », on veut implémenter en Python le tri fusion vu en cours. Comme indiqué dans ce dernier, bien que le pseudo-code de ce tri soit relativement simple, l'implémenter efficacement en Python l'est moins. Nous allons donc travailler sur trois versions successives de ce même tri en essayant de l'améliorer à chaque fois.

Version du cours

Nous commençons par implémenter la version étudiée en cours.

Question 1 En partant du pseudo-code présenté à la slide 8 du cours, écrire une fonction Python `fusion` prenant en paramètre deux listes Python déjà triées `l1` et `l2`. Cette fonction devra retourner une nouvelle liste Python de taille `len(l1) + len(l2)` tous les éléments triés de `l1` et `l2`. Rappel : Comme indiqué sur la slide 8 du cours, il faut porter une attention particulière aux indices qui parcourent les deux listes pour éviter les `IndexError`.

Question 2 En partant du pseudo-code présenté à la slide 10 du cours, écrire une fonction Python `tri_fusion` prenant en paramètre une liste Python `l`. Cette fonction retournera une nouvelle liste correspondant à la liste `l` triée. Vérifier le fonctionnement de l'algorithme sur la liste `l = [7, 5, 3, 0, 1, 6, 4, 2]`.

Question 3 Analogiquement à ce qui a été fait dans l'exercice 4 du TP1, mesurer le temps d'exécution de la fonction `tri_fusion` pour des listes de nombres aléatoires de taille croissante. Afficher la courbe des temps mesurés en fonction de la taille d'entrée. Enfin, afficher une courbe de la complexité étudiée en cours de ce tri afin de vérifier que les résultats mesurés coïncident avec la théorie.

Question 4 Nous avons jusque là étudié la complexité temporelle du tri fusion. Que pouvez-vous dire de la complexité spatiale de ce dernier ?

Version efficace en espace : le tri fusion en place

Dans le cours, nous avons vu que le tri fusion a une complexité temporelle en $O(n \log(n))$, mais il nécessite un espace mémoire supplémentaire proportionnel à la taille de la liste à trier (complexité spatiale en $O(n)$). Pour remédier à cela, il est possible de réaliser un tri fusion en place, c'est-à-dire sans créer de nouvelles listes, mais en modifiant directement la liste d'origine. Ce faisant, la complexité spatiale devient $O(1)$.

Au lieu de manipuler des sous-listes séparées de la liste originale pour chaque appel à la fonction `fusion`, nous allons travailler directement sur les indices de cette liste. Cela signifie que pour trier une partie de la liste, nous passerons les indices correspondant au début et à la fin de la portion à trier. Exemple : Si nous avons la liste `l = [7, 5, 3, 0, 1, 6, 4, 2]` et que nous appelons la fonction `tri_fusion_en_place`, deux sous-listes seront triées : celle

correspondant aux indices 0 à 3 (soit `[7, 5, 3, 0]`) et celle correspondant aux indices 4 à 7 (soit `[1, 6, 4, 2]`).

Question 5 Écrire une fonction `tri_fusion_en_place` prenant en paramètre une liste `l` et deux indices `i` et `j`. La fonction devra trier en place les éléments de `l` entre les indices `i` et `j` inclus.

Question 6 Écrire une fonction `fusion_en_place` prenant en paramètre la liste `l` et trois indices : `i` (début de la première sous-liste), `milieu` (début de la seconde sous-liste), et `j` (fin de la seconde sous-liste). L'objectif est de fusionner les deux sous-listes de manière ordonnée sans utiliser de mémoire supplémentaire. Pour ce faire, vous utiliserez des rotations pour placer les éléments au bon endroit dans la liste.

Indication : La stratégie consiste à parcourir la première et la deuxième sous-liste et, si un élément de la deuxième sous-liste est plus petit qu'un élément de la première, il doit être "inséré" dans la première partie en faisant des rotations des éléments situés entre les deux sous-listes. Cela vous permettra de « déplacer » des éléments de la seconde sous-liste vers la première sous-liste.

Question 7 Écrire une fonction `rotation_droite` prenant en paramètre une liste `l` et deux indices `i` et `j`. La fonction doit effectuer une rotation vers la droite des éléments situés entre les indices `i` et `j`, en déplaçant l'élément à l'indice `j` en position `i`.

Exemple : Pour la liste `l = [0, 3, 5, 7, 1, 2, 4, 6]`, une rotation vers la droite entre les indices `i=1` et `j=4` donnera `l = [0, 1, 3, 5, 7, 2, 4, 6]`.

Question 8 Faire une étude empirique de la complexité temporelle de l'algorithme via des mesures de temps. La complexité observée correspond-t-elle à la complexité attendue en $O(n\log(n))$? Si non, qu'est-ce qui, d'après-vous, pourrait expliquer cette différence ?

Vers une version efficace en temps et en espace

Les deux versions étudiées jusque là présentent des inconvéniants importants : la première a une complexité temporelle intéressante mais peut poser des problèmes de mémoire lorsqu'on lui demande de trier de grands tableaux, et la seconde passe outre ce problème de mémoire mais a pour conséquence de dégrader la complexité temporelle en $O\left(n^2\right)$. On voudrait donc avoir un algorithme de tri fusion qui permette à la fois d'avoir une complexité temporelle en $O(n\log(n))$, et une complexité spatiale constante.

Cette partie, plus difficile, est laissée en exercice et ne sera pas traitée en classe. Une implémentation possible est celle présentée dans [KPT96]. Le point clé de cette implémentation est l'utilisation d'une « zone tampon »(working area dans le travail original), qui est une partie déjà triée du tableau dont on se sert afin d'y stocker des valeurs lors du tri de l'autre partie, pas encore triée, de celui-ci.

Références

[KPT96] Jyrki Katajainen, Tomi Pasanen, and Jukka Teuhola. Practical in-place mergesort. Nord. J. Comput., 3(1):27–40, 1996.